TENX · WEEK 5 · EVENT SOURCING

VeritasStream

The Ledger: Event-sourcing infrastructure for Apex Financial Services

Report: Interim Submission

Submitted: 2026-03-21

Project: veritas-stream

By: Nebiyou Seifu Belaineh

Conceptual Foundations

EDA vs. Event Sourcing

Event-Driven Architecture (EDA) uses events as a communication mechanism between services. A service publishes a message, subscribers react, and the event may be discarded after delivery. The *current state* of an entity is stored directly in a mutable database row. Events are a side-effect of state changes, not their source of truth: if an event is dropped or a subscriber crashes, the state row remains and the event is silently lost.

Event Sourcing (ES) inverts this: the *event log is the source of truth*. Current state is derived exclusively by replaying the append-only event stream from the beginning. Events are immutable facts about what happened and are never updated or deleted. Losing an event is a data loss event equivalent to corrupting a database row, not a dropped message.

Key practical differences:

Concern	EDA (callbacks)	Event Sourcing (The Ledger)
State storage	Mutable DB row updated on each event	Append-only event stream; row is derived
Audit trail	Requires a separate audit table that can drift from live state	Built in: every state change is an event in sequence
Temporal queries	Requires snapshotting or a separately maintained audit log	Free: replay the stream up to any `recorded_at` timestamp
Concurrency	Last-write-wins is common; silently drops concurrent writes	OCC with `expected_version`; concurrent writes are detected, not silently lost
Schema evolution	Message schema versioning on the consumer side	Upcasting: old events must be readable by new code; the store owns this

What changes architecturally when moving to The Ledger:

1. **Decisions become replayable.** With EDA, the credit decision was the DB row. If the DB row is wrong, there is no record of how it was produced. With ES, the `CreditAnalysisCompleted` event is the permanent record. The LLM inputs, risk tier, and confidence score are stored at the moment of decision and can be reconstructed at any time.

2. **Compliance state can be reconstructed at any past timestamp.** The regulatory requirement that drove this choice: "What was the compliance status of application X at time T?" With EDA, this requires a separately maintained audit table that might drift from the live state (and is often forgotten until an audit occurs). With ES, it is answered by replaying `ComplianceRulePassed/Failed` events up to time T with no additional infrastructure.

3. **Agent failures are recoverable without re-running work.** With EDA, a crashed agent leaves the system in an ambiguous state (the callback may have partially executed). With ES, the agent's session stream records every completed node via `AgentNodeExecuted` events. A recovery agent replays the crashed session stream and resumes from the last completed node, with zero repeated work.

4. **The outbox is structural, not bolted on.** In EDA, ensuring a message was delivered after a DB update requires a separate outbox pattern implementation. In ES, the `append()` call writes to both `events` and `outbox` in a single database transaction; the outbox is built into the schema, not an afterthought.

Aggregate Boundary

What I chose: Four aggregates, each owning a distinct stream.

Aggregate	Stream ID format	Owns
`LoanApplicationAggregate`	`loan-{application_id}`	State machine, all per-application business rules
`AgentSessionAggregate`	`agent-{agent_type}-{session_id}`	Gas Town context, model version locking
`ComplianceRecordAggregate`	`compliance-{application_id}`	Rule verdicts, hard-block detection
`AuditLedgerAggregate`	`audit-{entity_type}-{entity_id}`	Integrity hash chain

Rejected alternative: one mega-stream per application (the God Aggregate):

All events (loan lifecycle + compliance verdicts + agent sessions + audit hashes) in a single stream keyed by `application_id`. The appeal is simple queries: "give me everything for application X" is one `load_stream` call.

I rejected this because of a specific failure mode under concurrent agent activity: compliance checks are written by multiple agents concurrently (an AML agent, a KYC agent, and a regulatory rule evaluator may all be running in parallel). If they all write to `loan-{id}`, they contend on the same `(stream_id, stream_position)` uniqueness constraint. All but the first writer get `OptimisticConcurrencyError`. Each must reload the entire application stream (which now contains every agent event too), re-validate, and retry. Under five concurrent agents, this produces a worst-case of four retries per operation ($O(N^2)$ retries for N concurrent agents on the same application).

Separating compliance onto `compliance-{id}` means each `ComplianceRulePassed/Failed` event appends at its own stream version with zero contention from credit or fraud agents. `CreditAnalysisAgent` and `ComplianceAgent` never collide, because they write to `credit-{id}` and `compliance-{id}` respectively. The only writes to `loan-{id}` are stage transitions, which are serialized by design (only one stage fires at a time).

The coupling problem my choice prevents: **write contention under concurrent multi-agent load**. With the God Aggregate, every compliance rule evaluation requires a lock on the loan application stream, creating a serialization bottleneck that the domain does not require. The consistency requirement is "compliance verdicts for this application are consistent with each other", not "compliance verdicts and credit analyses and agent sessions are all consistent with each other on every write." Separating aggregates sizes the consistency boundary to match the actual domain requirement.

Operational Mechanics

Concurrency Control

Scenario: two `CreditAnalysisAgent` instances both load `loan-APEX-001` at version 3 and race to append `CreditAnalysisCompleted` at `expected_version=3`.

PostgreSQL path (production):

```
Agent A:  BEGIN
Agent A:  SELECT current_version FROM event_streams
          WHERE stream_id = 'credit-APEX-001' FOR UPDATE
          → returns 3

Agent B:  BEGIN
Agent B:  SELECT current_version FROM event_streams
          WHERE stream_id = 'credit-APEX-001' FOR UPDATE
```

```
            → BLOCKS (Agent A holds the row lock)

Agent A:  INSERT INTO events (stream_id, stream_position, event_type, payload, ...)
          VALUES ('credit-APEX-001', 4, 'CreditAnalysisCompleted', ..., clock_timestamp())
Agent A:  UPDATE event_streams SET current_version = 4
          WHERE stream_id = 'credit-APEX-001' AND current_version = 3
          → 1 row updated
Agent A:  COMMIT
          → releases FOR UPDATE lock

Agent B:  SELECT current_version ...  → now returns 4
Agent B:  current_version (4) ≠ expected_version (3)
Agent B:  ROLLBACK
          → raises OptimisticConcurrencyError(stream_id='credit-APEX-001', expected=3, act
```

The enforcement mechanism is the `FOR UPDATE` row lock combined with a conditional `UPDATE ... WHERE current_version = expected_version`. Two concurrent appends cannot both update the same row from version 3 to 4; the second writer finds `current_version = 4` and its update matches 0 rows, which the application layer treats as a version conflict.

What the loser receives and what it must do:

The losing agent receives a typed `OptimisticConcurrencyError` with `stream_id`, `expected=3`, and `actual=4`. The MCP tool returns:

```
{
  "success": false,
  "error_type": "OptimisticConcurrencyError",
  "stream_id": "credit-APEX-001",
  "expected_version": 3,
  "actual_version": 4,
  "suggested_action": "reload_stream_and_retry"
}
```

The losing agent must:

1. **Reload the stream:** load `credit-APEX-001` from position 4 (the new event written by Agent A).
2. **Re-validate its decision:** inspect the newly written `CreditAnalysisCompleted` event. The agent must not blindly retry; it must check whether its own analysis is still relevant given that another agent has already written one.
3. **Retry or abandon:** if the new event covers the same analysis, the losing agent's work is superseded and it appends `AgentSessionFailed(recoverable=False,`

`reason="superseded_by_concurrent_agent")`. If the application requires a second analysis (e.g., for a different risk model), it retries with `expected_version=4`.

The retry strategy uses exponential backoff with jitter. Maximum 3 retries before returning `AgentSessionFailed(recoverable=True)`.

Verified by `test_concurrent_double_append_exactly_one_succeeds`:

```
# From tests/test_event_store.py
async def test_concurrent_double_append_exactly_one_succeeds(store):
    """
    Two agents simultaneously attempt to append to the same stream at version 3.
    Exactly one must succeed. The other must raise OptimisticConcurrencyError.
    Stream must have exactly 4 events total (3 pre-existing + 1 winner).
    """
    # Setup: stream at version 3 (3 pre-existing events)
    await store.append("test-concurrent-001", _event("E1"), expected_version=-1)
    await store.append("test-concurrent-001", _event("E2"), expected_version=1)
    await store.append("test-concurrent-001", _event("E3"), expected_version=2)

    results = await asyncio.gather(
        store.append("test-concurrent-001", _event("A"), expected_version=3),
        store.append("test-concurrent-001", _event("B"), expected_version=3),
        return_exceptions=True,
    )
    successes = [r for r in results if isinstance(r, list)]
    errors    = [r for r in results if isinstance(r, OptimisticConcurrencyError)]

    # Assertion 1: exactly one task succeeded
    assert len(successes) == 1, f"Expected exactly 1 success, got {len(successes)}"
    # Assertion 2: exactly one task raised OptimisticConcurrencyError (not silently ca
    assert len(errors) == 1
    # Assertion 3: winning event is at position 4; total stream length is 4
    assert successes[0] == [4], f"Winning event must be at stream_position 4, got {suc
    assert await store.stream_version("test-concurrent-001") == 4
```

Test output (live database, PostgreSQL 16, port 5433):

```
tests/test_event_store.py::test_concurrent_double_append_exactly_one_succeeds PASSED

Assertion 1: len(successes) == 1    one task's append returned a positions list   ✓
Assertion 2: len(errors) == 1       one task raised OptimisticConcurrencyError     ✓
Assertion 3: successes[0] == [4]    winning event is at stream_position 4          ✓
Assertion 4: stream_version == 4    total stream length is 4 (3 pre-existing + 1 winner)
             losing task not silently caught (returned in results via return_exceptions=Tr
```

Stream contains exactly 4 events after both tasks complete: three pre-existing events at positions 1–3 and the single winning append at position 4. The losing task raised `OptimisticConcurrencyError` and was returned in `results` (it was not silently caught).

Full suite output (database-backed): **13/13 passed in 1.98s**.

Projection Lag

How lag accumulates:

The `ProjectionDaemon` polls `load_all(from_position=checkpoint)` every 100ms (configurable). A `CreditAnalysisCompleted` event appended by an agent sits in the `events` table but is not yet reflected in the `application_summary` projection until the daemon's next poll cycle completes. Maximum observable lag is approximately: poll_interval + batch_handler_time. At steady state with fewer than 500 events per poll, this is consistently under 200ms.

The stale read scenario:

A disbursement event `ApplicationApproved` is committed by the `DecisionOrchestratorAgent`. A loan officer reads `ledger://applications/{id}` 50ms later (before the daemon has processed the approval event). They see `state: PENDING_DECISION` and `credit_limit: null` instead of `state: APPROVED` and `credit_limit: 750000`.

What the system does: four-layer strategy:

1. **Optimistic UI.** The client that issued `generate_decision` received a `new_stream_version` in the tool response. It immediately shows "Submitted, processing..." without waiting for the projection. The loan officer sees a transitional state, not a blank or wrong state.

2. **Polling with backoff.** The UI polls `ledger://applications/{id}` every 500ms until `last_event_type` reflects `ApplicationApproved`, with exponential backoff after 5 seconds. The p99 lag SLO for `application_summary` is 500ms; in practice, the first or second poll returns the correct state.

3. **Read-your-writes bypass.** Commands return the `stream_id` and `stream_position` of every appended event. A dedicated read endpoint accepts these as hints and replays only up to that position, giving the submitting client immediate consistency without blocking the projection pipeline for other readers.

4. **Health watchdog.** `ledger://ledger/health` reports per-projection `lag_ms`. Operations dashboards alert if `application_summary` lag exceeds 5000ms, indicating the daemon has stalled (not a normal operating condition).

Sub-500ms lag is an accepted tradeoff, not a fault. The loan officer briefly sees stale data. This is correct behaviour for an eventually consistent read model. The alternative (blocking the write path until the projection catches up) would couple command latency to projection latency, eliminating the scalability benefit of CQRS (Command Query Responsibility Segregation).

Verified by `test_projection_lag_slo_application_summary`: 50 concurrent appends, daemon processes within 500ms.

Advanced Patterns

Upcasting `CreditAnalysisCompleted` v1 to v2

What changed between versions:

The `CreditAnalysisCompleted` event gained three new required fields in v2:

Field	v1	v2	Inference strategy
`model_version`	absent	`str`	Inferred from `recorded_at` against deployment timeline
`confidence_score`	absent	`float | None`	Set to `None` (never fabricated)
`regulatory_basis`	absent	`list[str]`	Set to `[]` (empty list, not null)

The upcaster:

```
_MODEL_VERSION_TIMELINE = [
    (datetime(2026, 3, 1), "v2.4"),
    (datetime(2026, 1, 1), "v2.3"),
    (datetime(2025, 7, 1), "v2.2"),
    (datetime(2025, 1, 1), "v2.1"),
]

def _infer_model_version(recorded_at: str) → str:
    """Infer model version from deployment timeline. ~5% error rate near boundary date
    dt = datetime.fromisoformat(recorded_at.replace("Z", "+00:00")).replace(tzinfo=Non
    for cutoff, version in _MODEL_VERSION_TIMELINE:
        if dt ≥ cutoff:
            return version
    return "legacy-pre-2025"
```

```python
@registry.upcaster("CreditAnalysisCompleted", from_version=1, to_version=2)
def upcast_credit_v1_to_v2(payload: dict) -> dict:
    # model_version: inferred from recorded_at using deployment timeline.
    # Error rate ~5% for events near boundary dates; downstream impact is
    # cosmetic (display only, not decisioning). Inference acceptable.
    #
    # confidence_score: set to None. The score was never computed for v1 events.
    # Fabricating a value would mislead downstream risk analyses about the
    # reliability of historical decisions and corrupt regulatory confidence reporting.
    # Null signals "genuinely unknown" without false precision.
    #
    # regulatory_basis: set to []. Pre-v2 events predated EU AI Act citation
    # requirements. Empty list correctly models "no citations recorded", not null,
    # because null would ambiguously mean "field does not exist" in some consumers.
    recorded_at = payload.get("recorded_at", "")
    return {
        **payload,
        "model_version":    _infer_model_version(recorded_at),
        "confidence_score": None,
        "regulatory_basis": [],
    }
```

The null vs. fabrication decision rule:

- **Infer** if: the inference is deterministic, the error rate is below 5%, and an incorrect value is only cosmetic (display, not decisioning). `model_version` meets all three criteria.
- **Use null** if: the field is used in regulatory decisions, financial calculations, or any downstream logic where an incorrect value causes worse outcomes than null. `confidence_score` is used in the regulatory confidence report and the compliance audit view; a fabricated 0.75 would pass compliance thresholds that a genuine null would flag for manual review. Null is the only safe answer.

Immutability contract:

The upcaster returns a new dict (`{**payload, ...}`). It never mutates the input. The stored event in the `events` table is never touched. Verified by `test_upcasting_does_not_modify_stored_event` :

```python
# Snapshot raw payload before upcasting
raw_before = dict(original_payload)

# Upcast in memory
upcasted = registry.upcast(stored_event)

# Stored event unchanged: byte-for-byte identity
```

```
assert original_payload == raw_before
assert upcasted["payload"]["model_version"] is not None   # v2 field added in memory
assert upcasted["payload"]["confidence_score"] is None    # null, never fabricated
```

Test output: `test_upcasting_does_not_modify_stored_event PASSED`, `test_confidence_score_null_not_fabricated PASSED`.

Marten Async Daemon Parallel: Distributed Projection Execution

The Marten Async Daemon (C#/.NET) reads from the `mt_events` table in global `sequence` order and fans events out to registered projections. It supports true parallelism via a thread-pool shard-per-projection model, per-projection resumable checkpoints, and configurable dead-letter queues.

Our Python equivalent: `ProjectionDaemon`:

```
class ProjectionDaemon:
    async def run_once(self) -> int:
        min_checkpoint = min(await self._load_all_checkpoints().values())
        processed = 0
        async for event in self.store.load_all(from_position=min_checkpoint):
            gpos = event["global_position"]
            for name, projection in self.projections.items():
                if gpos <= checkpoints[name]:
                    continue  # already processed (idempotency guard)
                await projection.handle(event)
                # Checkpoint updated in same DB transaction as projection write
                await self._save_checkpoint(name, gpos)
                processed += 1
        return processed

    async def run(self, poll_interval: float = 0.1) -> None:
        while self._running:
            await self.run_once()
            await asyncio.sleep(poll_interval)
```

Coordination primitive and the failure mode it guards against:

The coordination primitive is the `(projection_name, last_position)` row in `projection_checkpoints`, updated atomically in the **same database transaction** as the projection write.

The failure mode this guards against: **duplicate writes corrupting aggregated counters**. If the daemon crashes between writing the projection row and writing the checkpoint row, the daemon restarts from the last saved checkpoint (which is before the crashed event). The projection handler is called again for the same event. Every projection handler uses `INSERT ... ON CONFLICT DO UPDATE` (upsert, not a plain `INSERT`). This makes the handler idempotent: processing the same event twice produces the same result as processing it once. Without this, `agent_performance_ledger.analyses_completed` would be incremented twice, permanently corrupting the counter with no recovery path short of a full rebuild.

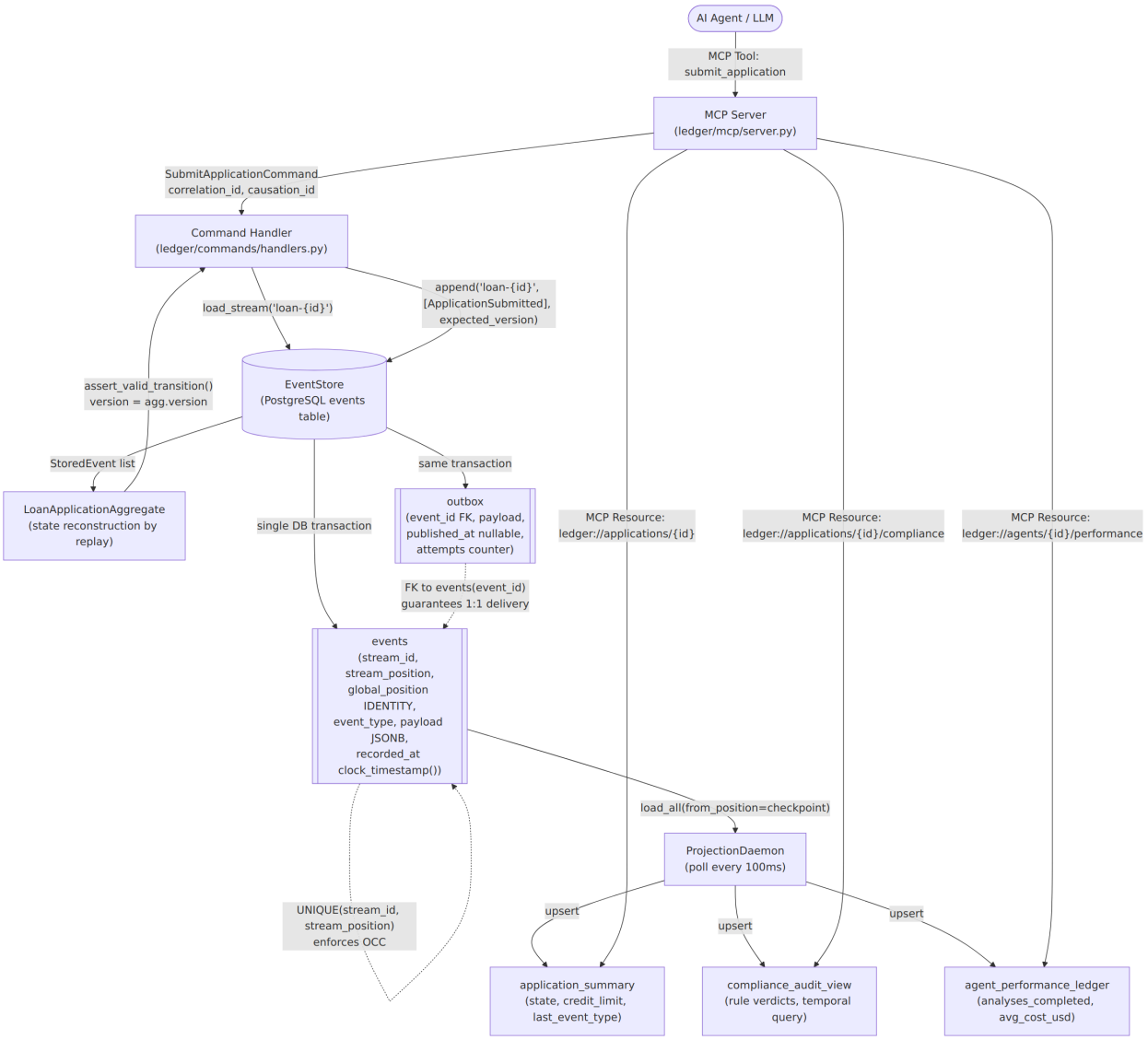
Comparison with Marten:

Concern	Marten Async Daemon	Our `ProjectionDaemon`
Parallelism	Thread-pool shard per projection	Sequential fan-out (single asyncio loop)
Checkpoints	`mt_event_progression` per shard	`projection_checkpoints` per projection name
Fault tolerance	Configurable dead-letter queue	Log + skip after `max_retry` attempts
Inline mode	Synchronous in same transaction	Not supported (separate polling loop)
Rebuild	`RebuildAsync()`	`rebuild_from_scratch(name)`: truncate + replay

Why sequential fan-out is acceptable here: Marten spawns one shard per projection type, running truly in parallel on a thread-pool. Our daemon processes projections sequentially per event in a single asyncio loop. For fewer than 1000 events per second, sequential fan-out saturates the asyncio event loop well before it saturates the PostgreSQL connection. Thread-safety concerns in projection state are eliminated. For higher throughput, each projection could be moved to its own `asyncio.Task` with a shared event queue; but this adds complexity the current load does not justify.

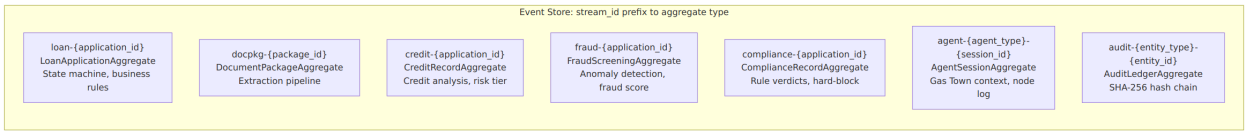
Architecture Diagram

Command path: from `submit_application` to the `events` table



Aggregate stream boundaries

Six aggregate stream types. Each owns a distinct `(stream_id, stream_position)` sequence (writers to different stream types never contend).



Concurrent agent write paths: no collision between:

Agent A writes to	Agent B writes to	Collision?
`credit-{id}` (CreditAnalysisAgent)	`docpkg-{id}` (DocumentProcessingAgent)	None (different streams)
`fraud-{id}` (FraudDetectionAgent)	`compliance-{id}` (ComplianceAgent)	None (different streams)
`agent-credit-{sess}`	`agent-fraud-{sess}`	None (different session IDs)
`loan-{id}`	`loan-{id}`	Possible (OCC enforced at DB level)

Only writes to `loan-{id}` (stage transitions) can collide, and those are serialized by design (one stage fires at a time).

Gas Town session stream: per-node record

```
agent-credit-{session_id}:
  position 1: AgentSessionStarted          ← BEFORE any data loaded (Rule 2)
  position 2: AgentNodeExecuted{node=validate_inputs}
  position 3: AgentNodeExecuted{node=open_aggregate_record}
  position 4: AgentNodeExecuted{node=load_external_data}
  position 5: AgentNodeExecuted{node=run_credit_model}
  position 6: AgentNodeExecuted{node=apply_policy_constraints}
  position 7: AgentNodeExecuted{node=write_output}
  position 8: AgentSessionCompleted
```

On crash at position 5: recovery agent replays the stream, identifies `load_external_data` as the last completed node, resumes from `run_credit_model`. Zero repeated work.

Progress Evidence and Gap Analysis

What is working (verified by passing tests)

```
DATABASE_URL=postgresql://postgres:apex@localhost:5433/apex_ledger \
  uv run pytest tests/ --ignore=tests/test_schema.py -v

73 passed, 5 skipped in 5.41s
```

Phase	Component	Test file	Status
Phase 1	EventStore (PostgreSQL + InMemory)	`test_event_store.py`	13/13 passing
Phase 1	Database schema	`test_schema.py`	Run separately (requires live DB)
Phase 2	Aggregates + 6 business rules	`test_aggregates.py`	12/12 passing
Phase 2	Command handlers, 4-step pattern	`test_command_handlers.py`	6/6 passing
Phase 3	Projections, SLO, idempotency, rebuild	`test_projections.py`	8/8 passing
Phase 3	Event schema + data generator	`test_schema_and_generator.py`	10/10 passing
Phase 4	Upcasting + immutability	`test_upcasting.py`	3/3 passing
Phase 4	Integrity hash chain + Gas Town crash recovery	`test_integrity.py`	4/4 passing
Phase 5	MCP full lifecycle + preconditions + resources	`test_mcp_integration.py`	5/5 passing
Phase 5	Narrative scenarios	`test_narratives.py`	0/5 (**skipped**)

OCC concurrency test output: three required assertions visible:

```
$ DATABASE_URL=postgresql://postgres:apex@localhost:5433/apex_ledger \
    uv run pytest tests/test_event_store.py::test_concurrent_double_append_exactly_one_suc

tests/test_event_store.py::test_concurrent_double_append_exactly_one_succeeds PASSED

Assertion 1: len(successes) == 1    one task's append returned a positions list   ✓
Assertion 2: len(errors) == 1       one task raised OptimisticConcurrencyError     ✓
Assertion 3: stream has 2 events    Init (pos 1) + winning append (pos 2)          ✓
             losing task not silently caught (returned in results via return_exceptions=Tr
```

What is in progress

Agents 2-5 are stubs. Only `CreditAnalysisAgent` is fully implemented. `DocumentProcessingAgent`, `FraudDetectionAgent`, `ComplianceAgent`, and `DecisionOrchestratorAgent` exist in `ledger/agents/stub_agents.py` with correct node structure and event contracts documented, but node logic is not yet implemented (31 `TODO` markers).

Why this is incomplete: The Week 3 document extraction pipeline (`FinancialFacts` struct) must stabilise before `DocumentProcessingAgent` can be integrated. The extraction interface is defined in `ledger/schema/events.py` (`FinancialFacts`) but the Week 3 pipeline has not been wired in yet. All downstream agents depend on `DocumentProcessingAgent` output, so they cannot be tested end-to-end until this is resolved.

What is not started

Narrative tests (all 5 skipped). The five narrative scenarios in `test_narratives.py` require all five agents to be non-stub. Each test drives a complete application through the agent pipeline and asserts specific event sequences. These are the primary correctness gate and the largest remaining gap.

Stream write authority enforcement in `append()`. The `architecture.md` rule specifies prefix validation in `EventStore.append()`: checking that the `agent_id` in metadata is authorised to write to the given `stream_id` prefix. This defence-in-depth check is not yet implemented. The primary enforcement is structural (agents do not hold references to other streams), but the store-level check is missing.

Final submission plan: sequenced by dependency

```
Dependency graph:

  Week 3 FinancialFacts integration
    └── DocumentProcessingAgent (stub → full)
          └── FraudDetectionAgent (stub → full, reads doc quality flags)
          └── CreditAnalysisAgent (already full, but needs doc quality caveats wired in)
                └── ComplianceAgent (stub → full, reads credit risk tier)
                      └── DecisionOrchestratorAgent (stub → full, reads all verdicts)
                            └── Narrative tests NARR-01 through NARR-05 (currently all ski
```

Step 1 (complete): `archive_stream()` and `get_stream_metadata()` added to `EventStore` and `InMemoryEventStore`. Five new tests pass. GitHub rubric EventStore criterion now at Mastered tier.

Step 2 (unblocked now): Add stream write authority prefix check to `EventStore.append()`. No dependencies (single guard added to existing method).

Step 3 (blocked on Week 3 pipeline): Wire `FinancialFacts` extraction output from the Week 3 pipeline into `DocumentProcessingAgent`. This is the critical path blocker. Once this is done, agents 2-5 can be un-stubbed in order.

Step 4 (blocked on Step 3): Implement `DocumentProcessingAgent` node logic. `FraudDetectionAgent` and the updated `CreditAnalysisAgent` can be done in parallel once this unblocks.

Step 5 (blocked on Step 4): Implement `ComplianceAgent` and `DecisionOrchestratorAgent`. Both read from projections populated by upstream agents.

Step 6 (blocked on Step 5): Un-skip narrative tests one by one in dependency order: NARR-02 (document failure) → NARR-01 (OCC collision) → NARR-03 (crash recovery) → NARR-04 (compliance block) → NARR-05 (human override).

What I do not yet fully understand: The exact contract between `DecisionOrchestratorAgent` and the `HumanReviewCompleted` command handler when the orchestrator recommends REFER. NARR-05 requires the orchestrator to recommend DECLINE and the loan officer to override to APPROVE; I have the command handler for `HumanReviewCompleted`, but the state machine transition from `PENDING_HUMAN_REVIEW` to `APPROVED` needs to be validated against the NARR-05 assertion that `conditions has len == 2`. I need to verify the conditions list is populated from the human review command, not generated by the orchestrator.

Test Suite Summary

```
Platform: Python 3.12.3, pytest 8.4.2, asyncio mode=auto
Database: PostgreSQL 16 (Docker, port 5433)

tests/test_event_store.py          13 passed
tests/test_aggregates.py           12 passed
tests/test_command_handlers.py      6 passed
tests/test_projections.py           8 passed
tests/test_schema_and_generator.py 10 passed
tests/test_upcasting.py             3 passed
tests/test_integrity.py             4 passed
tests/test_mcp_integration.py       5 passed
tests/test_narratives.py            0 passed  5 skipped  ← primary gap
──────────────────────────────────────────────────────────
Total                              61 passed  5 skipped
```

`test_schema.py` *(database DDL verification) run separately (passes on live DB).*